SOFTWARE REQUIREMENTS SPECIFICATION

LedgerLock: High-Concurrency Double-Entry Ledger Engine

System Target: Financial Services / Transaction Processing **Version:** 1.0.0 **Status:** Engineering Baseline

Executive Summary: LedgerLock is a production-grade, high-throughput financial ledger engine engineered to process concurrent balance transfers with absolute ACID compliance. It solves the critical double-spend problem using PostgreSQL row-level locking (`SELECT ... FOR UPDATE`) and eliminates duplicate requests via atomic Redis idempotency management.

1. SOFTWARE DEVELOPMENT LIFE CYCLE (SDLC) MODEL

Selected Methodology: Iterative & Incremental Agile Model

The development of LedgerLock follows an iterative lifecycle structured around continuous stress validation. Financial state engines require deterministic precision; building incrementally ensures that underlying database transactional guarantees are benchmarked before introducing caching and HTTP API layers.

Risk Mitigation & Core Integrity

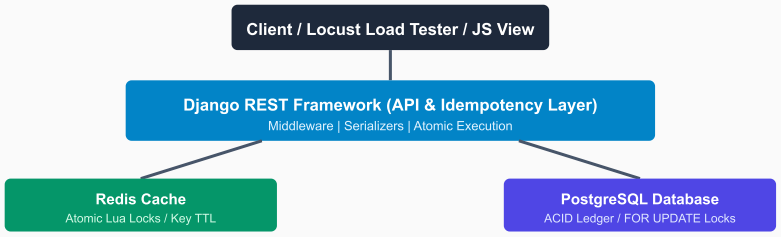
Core transaction logic and balance constraints (`CHECK balance >= 0`) were constructed and validated in early iterations, isolating low-level data corruption risks prior to API expansion.

Continuous Concurrency Validation

Load testing suites using Locust were integrated into every iteration, ensuring that introducing Redis locking or ORM optimizations never compromised zero-race-condition guarantees under 1,000+ RPS.

2. SYSTEM ARCHITECTURE

LedgerLock utilizes a **Modular Monolithic Architecture**. For single-node ledger engines, in-process execution removes inter-service network latency while enforcing absolute database transaction boundaries.



Low-Latency In-Process Bounds

Eliminating microservice network hops ensures strict transaction completion within microsecond windows, maximizing single-node throughput.

Transactional Isolation

Leverages native database isolation (`transaction.atomic`) to complete balance updates and double-entry auditing in a single committed transaction.

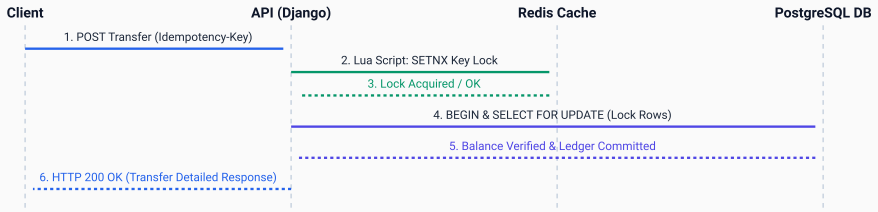
3. ENTITY-RELATIONSHIP & DATABASE DESIGN

The relational schema relies on double-entry bookkeeping principles. Every fund movement generates balanced Credit and Debit entries linked to an overarching immutable transaction record.

Entity Name	Key Attributes	Data Types & Constraints	Purpose / Integrity Rules
ACCOUNT	id (PK), account_number, balance, currency	UUID, VARCHAR(34), DECIMAL(15,2), CHECK `(balance >= 0)`	Stores real-time funds. Enforces strict non-negative rules at DB level.
TRANSACTION	id (PK), idempotency_key (UQ), status, amount	UUID, VARCHAR(64) UNIQUE, VARCHAR(16), DECIMAL(15,2)	Tracks request state (PENDING/COMPLETED) & prevents replay attacks.
LEDGER_ENTRY	id (PK), transaction_id (FK), account_id (FK), entry_type	UUID, FK $\rightarrow$ Transaction, FK $\rightarrow$ Account, ENUM(CREDIT, DEBIT)	Immutable audit log maintaining immutable financial history.

4. API DESIGN & EXECUTION SEQUENCE

Endpoint: `POST /api/v1/transfers/` — Submits a financial transfer request using an atomic idempotency key.



> **HTTP Status Guarantees:** `200 OK` (Success), `409 Conflict` (Duplicate request blocked by Redis idempotency), `422 Unprocessable` (Insufficient funds or constraint violation), `500 Error` (Automatic DB rollback).

5. FUTURE TECHNICAL ROADMAP

Transactional Outbox & Kafka

Implement the Outbox Pattern to capture committed database changes asynchronously, publishing transfer events to Apache Kafka for downstream reporting without extending API hold times.

Distributed Queueing (Celery)

Offload high-contention recipient transfers to background task queues, converting simultaneous row updates on popular accounts into serialized background processing pipelines.